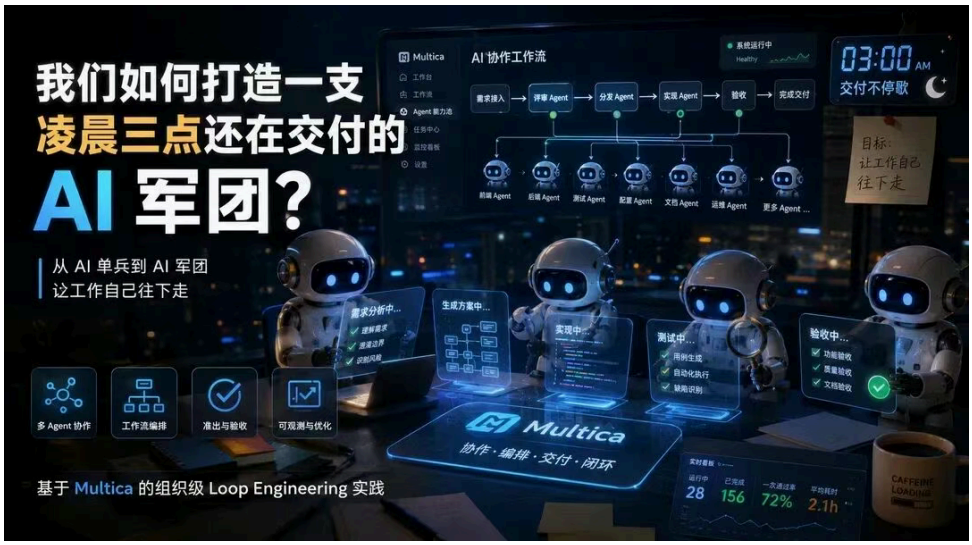
【揭秘】如何打造一支凌晨3点还在交付的AI军团

腾讯程序员 腾讯技术工程 2026年7月10日 02:36 广东 163人



作者：胡峻玮&姚广龙

> AI 已经让很多人成为”超级个体”，但一件完整工作并没有因此自动变快。AI 帮我写完了需求草稿、帮我定位了 Bug、帮我生成了一批用例、帮我整理了一份配置——AI 让每个人都成为了"超级个体"，却依然需要由“人”来作为主导推动。一件完整的工作仍要靠人一步步看、一步步转、一步步推：需求等人评审，Bug 等人接手，上下文在一次次转述里丢失。我们意识到，真正的瓶颈不在"每个人会不会用 AI"，而在“没有为 AI 的模式去设计一套新的工作方式”。这次基于 Multica，我们搭出了这支 AI 协作军团的雏形——这是我们对"组织级 Loop Engineering” 的一次思考实践。

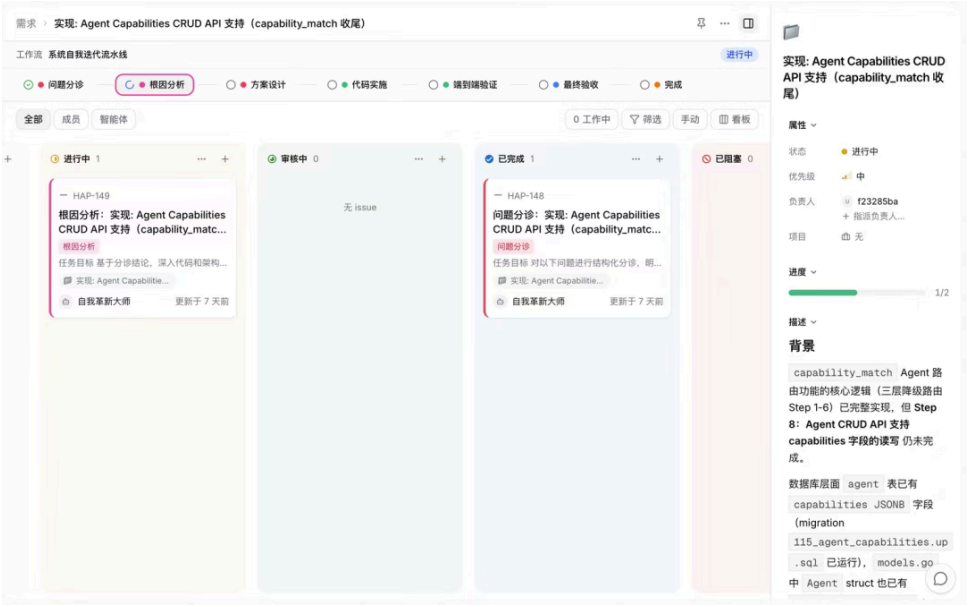


凌晨三点，没有人在群里催进度，也没有人在电脑前等结果。

但第二天早上当你打开平台，已经能看到一些工作进度发生了变化：一条需求走完了评审和分发，进入实现和验收入口；一个测试提的 Bug 被接手处理，修完后回到测试侧继续验收；平台自己的一个能力改进项，也跑完了分析、修改和验证，等人确认；历史问题池里那些过去总是排不上期的小问题，也开始被定期捞出来处理。

这些事情当然是 Agent 在做。

但真正的变化不在于某个 Agent 半夜多跑了一次，而在于一件事进入系统后，不需要再等人逐步叫醒下一个角色。系统会继续找到下一步、找到合适的 Agent、带着上下文往下推，直到走到通知、验收或返工的位置。



过去这一年，我们其实已经攒了不少 Agent。有的能评审需求，有的能分发任务，有的能改配置，有的能修 Bug，有的能写代码。

它们的单点能力都已经不弱——像是一群刚入职的优秀实习生：让他们做手上这一摊事，做得不错；但你不能指望他们自己知道这件事什么时候该开始、做完之后该交给谁、卡住了该找谁兜底。

更现实的是，这些"实习生"还分散在不同的角色手里。产品用自己的 AI，研发用自己的 AI，测试用自己的 AI，运营用自己的 AI。每个角色都变强了，但一件事从开始到完成，仍然要靠人一步步看、一步步转、一步步推。

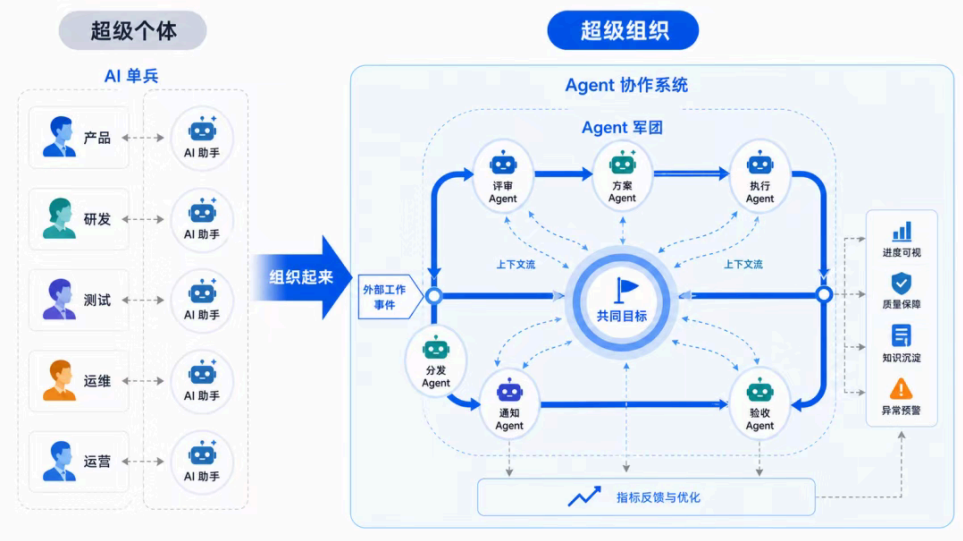
所以这次我们真正想试的，不是再训练一个更强的"实习生"，而是看看能不能**教这群实习生一起把一件事做成**——一条需求进来，不再只是等人看到后才去调用评审 Agent；一个 Bug 被提交，不再只是等研发逐单接手；一个平台待办出现，也不一定要先等人排期、拆解、推动。只要这件事边界清楚、目标明确、结果可验收，就让它进入这套机制，由不同 Agent 围绕同一个目标协作推进，最后把结果交给人确认。

这件事如果说到底是什么，我们的判断是这样：

> **AI 转型的下半场，不是让每个人都用上 AI，而是教 AI 怎么真正学会“工作”。**

过去，AI 更多是在帮助每个人成为"超级个体"。

这一次，我们想再往前走一步：看看一组 Agent 被组织起来之后，能不能长出"超级组织"的雏形。



一、AI 进了岗位，但还没有进入协作链路

每个角色都配上 AI 之后，组织效率却没有因此发生质变——卡点到底在哪里？

先回到那条最常见的工作流转链路：

```
产品理解需求
→ 评审判断合理性
→ 负责人分发任务
→ 研发实现
→ 测试验证
→ 负责人验收
→ 发布或关闭
```

AI 出现后，最自然的做法，是给链路上的每个角色都配上 AI。这当然有效，但改变的只是**每个节点的效率**，并没有改变整条链路的协作方式——工作仍然要等人看到、等人判断、等人分发、等人转述、等人验收。很多时候真正消耗时间的，不是某一步执行本身，而是中间的等待、交接、确认和返工。

具体来看，旧模式下有三处典型卡点。

第一，工作推进仍然依赖人的在线状态。 过去 Agent 会干活，但要等人叫它。需求来了，要有人看到；评审完了，要有人转述；该谁执行，要有人分发；做完以后，还要有人通知负责人验收。只要人不在，这条链路就容易停住。

这也是"人下班，Agent 也下班"的本质：不是 Agent 没有能力，而是旧模式里，Agent 的启动、衔接、推进和验收都依赖人。

第二，旧角色边界限制了 AI 扩大的能力边界。 AI 已经扩大了每个人能做的事。一个研发可以借助 AI 更快理解需求；一个测试可以借助 AI 判断问题可能出在哪里；一个运营可以借助 AI 分析配置规则；一个产品也可以借助 AI 补充验收标准。

但如果组织协作仍然严格按照过去的角色边界推进，那么 AI 放大的个人能力又会被旧流程重

新压回原来的格子里。

> **AI 扩大了个体能力，但旧流程仍然要求每个人只接自己那一段。**

第三，很多关键上下文丢在角色交接里。 旧流程里，很多信息并不稳定地存在于系统里，而是散落在角色之间的转述中。比如评审时真正担心的风险、分发时为什么判断给这个模块、执行时发现的边界条件、Agent 输出里哪些内容可信、验收不通过的真实原因，以及返工时应该重点改哪里。

这些信息在一次次交接中会丢失。但 Agent 要连续完成一件事，最需要的恰恰是这些上下文。如果上下文仍然靠人来读、来解释、来转述，那么 Agent 就很难真正接力——它们只是各自完成一小段任务，链路仍然要靠人粘起来。

回到那个"实习生"的比喻：我们给每个角色都配了能力不弱的实习生，但**没有人教过他们怎么和别人一起工作**。所以哪怕每个实习生都比过去更利索了，整条链路并不会因此自动变快。

> **AI 只提效了节点，没有提效整条协作链路。**



二、所以我们想验证的，是另一件事

这次实践想验证的，不是"某个 Agent 能不能做某一步"，而是更进一步的问题：

> 一段边界清晰、可验证的组织工作，能不能不再按照人类角色接力的方式推进，而是由一组 Agent 在系统里围绕同一个目标协作完成？

我们考虑过的几条路

要让这件事发生，路径并不止一条。动手之前，我们在三个方向上权衡过：

路径 A：造一个更强的"超级 Agent"。 让一个端到端的 Agent 自己看懂需求、自己拆解、自己执行、自己验收。这条路看起来最酷，但问题也最直接：单一 Agent 的可控性、可观测性、责任边界都不好处理；一旦它中间错了，几乎没有干预点。对一件涉及多个角色、多个外部系统的真实工作来说，这一步迈得太大。

路径 B：直接设计一套 AI 原生的协作流程。 不参考人类现有的流程，从 AI 协作的特性出发，重新组织"目标 → 计划 → 执行 → 验证 → 验收"。这条路是我们真正认为更有想象空间的方向，但它有一个现实问题：没有人能拍脑袋设计出最优的 AI 原生流程。如果没有第一阶段的真实运行数据，所谓的"AI 原生"很容易变成另一种凭直觉的设计。

路径 C：先把人类已经验证过的流程 Agent 化。 选择一类边界清晰、已经被人类协作跑通过的工作，把其中每个角色背后的执行者换成 Agent，让工作流来组织它们接力。它不是终局，但它能最快跑起来，能最快暴露真实问题，也能为后面任何方向积累数据。

我们最后选了路径 C，作为第一阶段的策略：**先把人类流程 Agent 化**。

不是因为它最理想，而是因为它最务实——它**足够接近原有流程**，容易落地，容易对齐责任边界，也容易被业务验收；同时它又**真的把多个 Agent 串了起来**，会逼出协作系统该有的所有边界问题。

第一阶段的目标不是一上来就设计一个终局式的 AI 原生流程，而是先把一个更基础的问题回答清楚：

> 一条原本需要多人接力的链路，能不能被一组 Agent 接起来跑？

三、技术底座：把 Multica 改造成 AI 军团的指挥中枢

策略定了，接下来要选起点。我们没有从零搭一个新平台，而是选择了 **Multica** 这个开源项目的能力作为技术底座。

Multica 不是最终答案，而是一个比较合适的起点。它已经具备 Agent、Issue、Runtime / Daemon、Agent Task、工作空间、基础任务分发和执行链路等能力，能比较自然地表达一个基础模式：

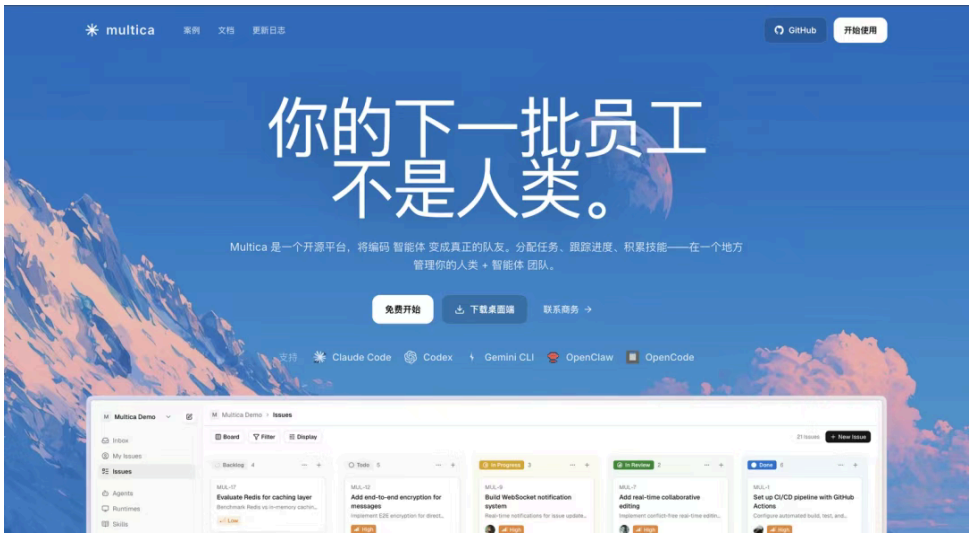
```
Issue → 分配给 Agent → Agent 执行 → 回写结果
```

这对单个 Agent 执行任务是够用的。但当我们想承接一条真实工作链路时，它还不够。

真实工作不是"一个 Issue 交给一个 Agent"就结束。它可能要先评审，再分发，再执行，再通知，再验收；有时要并行处理；有时 Agent 会 blocked；有时执行完成后还要外部系统验收；有时验收不通过，要回退到中间节点重做。

所以第一阶段二次开发的核心，不是给 Multica 加几个页面，而是把它从一个偏 **Agent 任务管理** 的系统，扩展成一个能支撑 **多 Agent 协作工作流** 的底座。

Multica 项目链接：https://github.com/multica-ai/multica



要让一组 Agent 在系统里协作起来，不是加几个工作流页面就能完成的。它要先回答三个最基础的问题：

```
平台能不能调得动 Agent?
平台能不能表达一条工作流?
平台能不能和外部系统完成交接?
```

这三件事构成了 Agent 协作系统的基础骨架。下面分别展开。

3.1 第一根骨架：把分散 Agent 变成平台可调度的能力池

第一阶段之前，我们已经有不少 Agent，但它们是分散的。有的在个人电脑上，有的在云主机上，有的在外部平台上，有的是某个业务方向的专项 Agent。

这些 Agent 各自有能力，但平台并不知道：有哪些 Agent、它们能做什么、现在能不能用、该怎么调用、这一步该派谁。

如果平台不知道这些，Agent 再多也只是散点能力。

所以第一步不是先画工作流，而是让 Agent 进入平台视野，变成平台可调度的能力池。

第一阶段，我们把平台内 Agent、本地 Agent、云主机 Agent、KNot、CodeBuddy 等不同来源的 Agent 接入到统一管理视图中。调度策略覆盖了"指定 Agent / 上一步指定 / 能力匹配 / 兜底解析"四种方式，从静态到动态都能落。

这一步不是单纯做一个 Agent 列表，而是把分散在个人和外部平台里的 AI 能力，变成平台可调度的执行能力。



3.2 第二根骨架：把人的流程经验变成可运行工作流

光有 Agent 还不够。

如果仍然靠人决定什么时候评审、评审完找谁、分发完谁执行、执行完谁验收、失败后回哪里，那协作模式没有变化。

所以第二根骨架是 **工作可编排**。

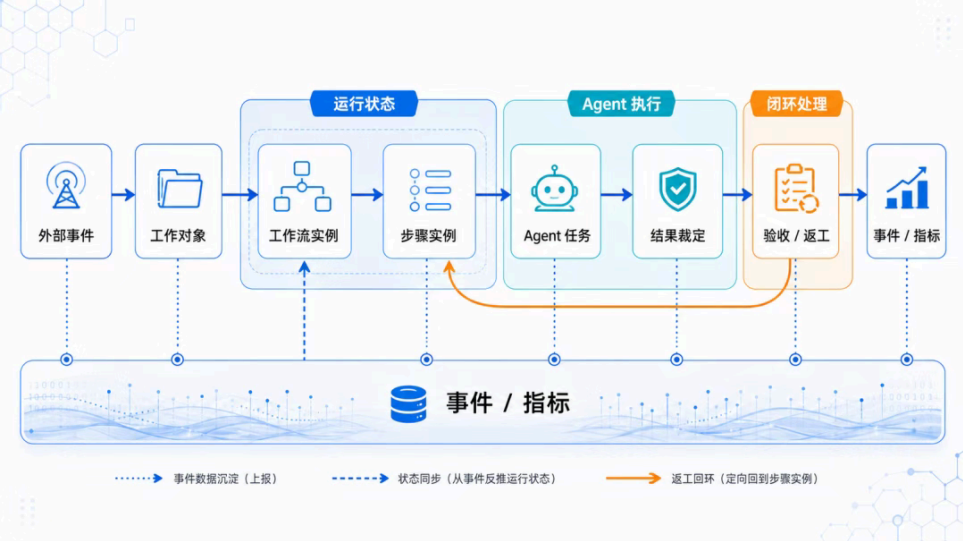
我们补出了工作流运行时能力：Workflow Template、Node、Edge、Step Instance、Issue / Agent Task、节点状态流转、End Node、Acceptance 等。它们负责把过去人脑里的流程经验，变成系统可以运行、追踪和恢复的流程结构。

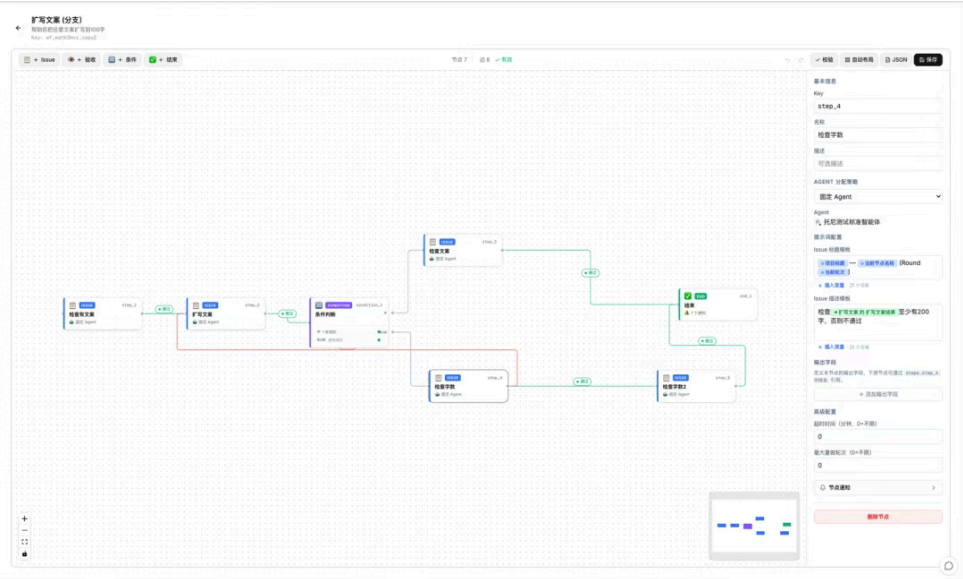
比如一类复杂流程的工作，可以表达为：

```
评审 → 技术方案 → 配置变更 → 模板变更 → 代码变更 → 通知完成
```

更简单的标准流程则只需要其中几个节点的子集。它们复用的是同一套模板、节点、边和 Step Instance 机制。节点被激活后，系统创建对应 Issue / Agent Task，并分配给合适 Agent 执行；Agent 提交结果后，工作流根据状态推进下一步。

这一步让"人知道下一步怎么走"，变成"系统知道下一步怎么走"。





3.3 第三根骨架：让真实工作进来，完成状态能出去

平台不能替代所有外部系统。

需求、缺陷、验收、发布系统已经存在。我们要做的不是把所有工作搬进一个新平台，而是让外部系统在合适的状态把工作交给平台，平台组织 Agent 执行，执行完成后再把结果或状态交回去。

所以第三根骨架是 **外部可交接**。它要回答四件事：外部工作如何进入平台、平台完成后如何通知负责人、外部验收结果如何回到平台、外部系统如何理解当前进度。

以需求系统为例，当需求进入可执行状态后，外部系统触发 Hook，把需求标题、描述、负责人、来源链接和 `template_key` 推给平台。平台创建工作对象并启动对应工作流。工作流完成后，负责人收到通知，并在平台或外部系统完成验收；验收通过则关闭，驳回则回到指定节点返工。

这一步让平台成为 Agent 执行层，而不是另一个孤立工单系统。

走到这里，Multica 不再只是分发任务的看板，而是一支 AI 军团可以在其上排兵、接力、验收和返工的指挥中枢。

四、从能跑到可用：真实运行里补出的复杂能力

有了 Agent 可调度、工作可编排、外部可交接这三根骨架，系统就具备了跑起来的基础。

但跑起来只是第一步。真实场景很快会暴露出 AI 工作流和传统工作流不一样的地方：Agent 输出不稳定，复杂工作不一定是线性的，Agent 完成不等于业务完成，流程可能静默卡住，验收不通过不能靠人重新解释一遍，跑完一批后还要知道哪里应该优化。

> 正向链路跑通只证明"能跑"，失败路径能处理才证明"可用"。

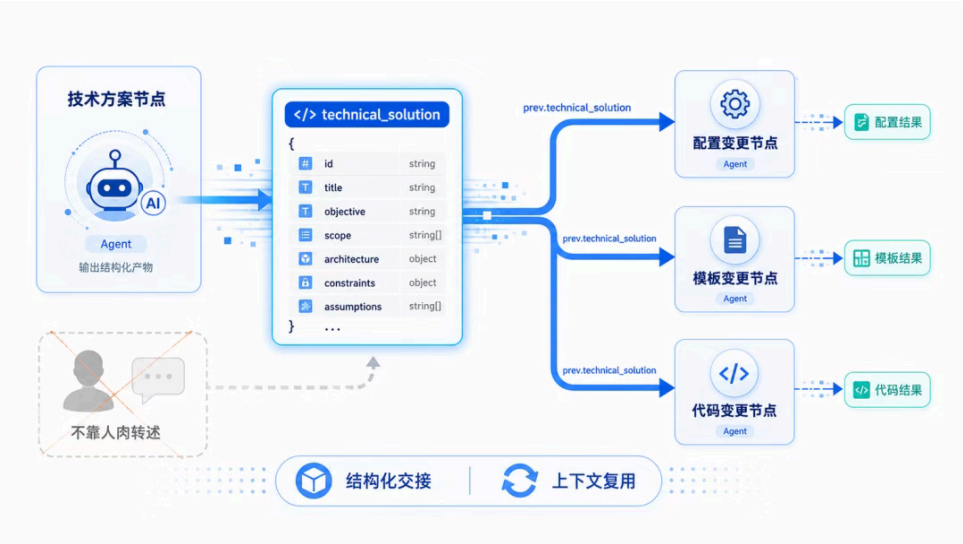
下面六类能力不是为了让系统“看起来”更完整，而是真实运行中被一个个找到的必要能力。

4.1 Agent 输出不稳定：准出字段和 Verdict

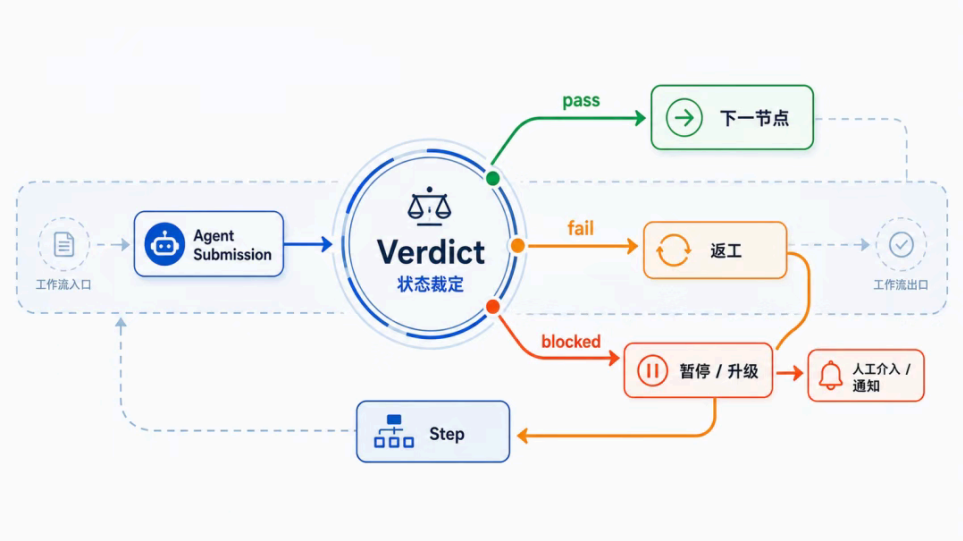
传统程序节点的输入输出通常比较稳定，但 Agent 节点不是这样。

它可能输出格式不稳定，漏掉下游需要的信息；也可能自然语言里说了很多，但系统无法稳定消费；它可能失败了但表达得很模糊，也可能 blocked 了但没有形成明确状态。

所以 AI 工作流不能假设每个节点天然可靠。我们用一组结构化字段约束 Agent 输出：**业务产物**承载真正要交付的内容，**流程裁定**（pass fail blocked）告诉系统下一步该往哪里走，**根因解释**和**置信度**则把"为什么这么判断"留在链路里，让下游不必重新猜测。



在这套字段之上，每一次 Agent 执行都会落成一次 Submission，并由系统派生出统一的 Verdict 状态。下游节点不再去解析自然语言，而是直接消费 Verdict 和准出字段——无论这一步交给的是哪一个 Agent。



4.2 线性链路不够：Fan-out 并行与收敛

标准工作可以按线性流程推进，但真实工作不一定这么简单。

一个需求可能同时涉及前端、后端、配置、模板、测试等多个方向。如果仍然让一个 Agent 或一个节点串行做完，效率低，也容易把多个方向的上下文混在一起，导致执行结果不稳定。

所以我们补了 **Fan-out 并行与收敛能力**。

上游节点可以拆出多个子任务，系统根据这些子任务动态创建并行子步骤。每个子步骤独立生成 Issue / Agent Task，分配给不同 Agent 执行；所有子步骤完成后，父步骤再 AND 收敛并推进下一节点。子任务失败时，可以通过 fail / blocked / rework 等策略决定兄弟任务的命运。

Fan-out 不是为了并行好看，而是让复杂工作不必被硬塞给一个 Agent。

4.3 Agent 完成不等于业务完成：通知、验收和返工

Agent 说"做完了"，不代表业务真的完成。

业务完成往往还需要负责人确认、外部系统验收、发布、Review、测试回归或需求方确认。如果把 Agent 完成直接当业务完成，风险很大。

所以第一阶段增加了 End Node、完成通知和 Acceptance 机制。

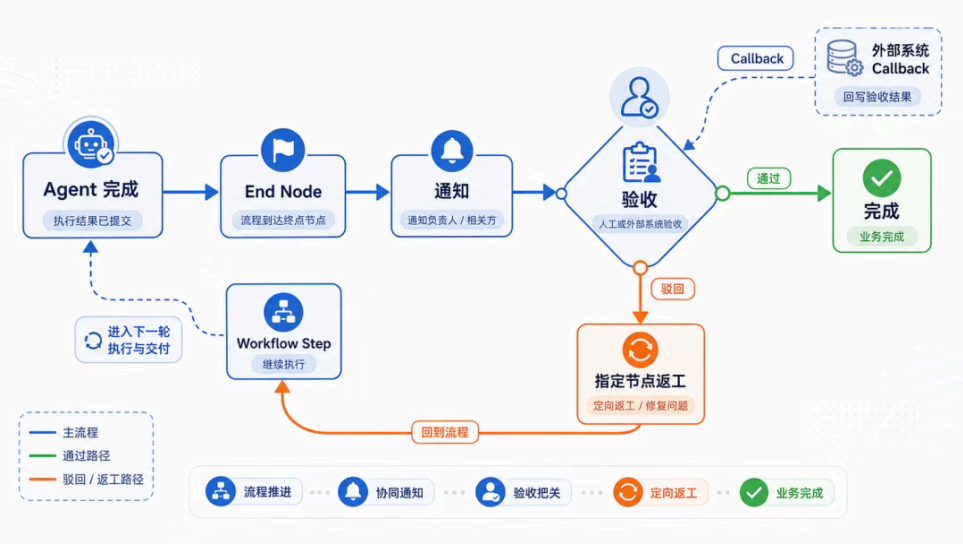
当工作流到达结束节点时，系统可以发布完成通知；如果需要验收，则进入 Acceptance 状态。负责人可以通过，也可以驳回到指定节点，并带上驳回原因。新一轮执行会带着 rework context 重新生成任务。

这里有两个关键判断。

第一，**Agent completed 不等于 business completed**。执行完成和业务完成必须拆开。

第二，**返工应该定向，而不是全链路重跑**。验收不通过时，应尽量回到真正出问题的节点，并把驳回原因带入新一轮执行，而不是让人重新解释一遍所有上下文。

这一步让 Agent 工作流具备了交付闭环：执行、通知、验收、驳回、返工。





4.4 没人盯时流程卡住：自愈与显式阻塞

要让 Agent 在没人持续盯着的时候继续干活，最怕的不是失败，而是静默卡住。

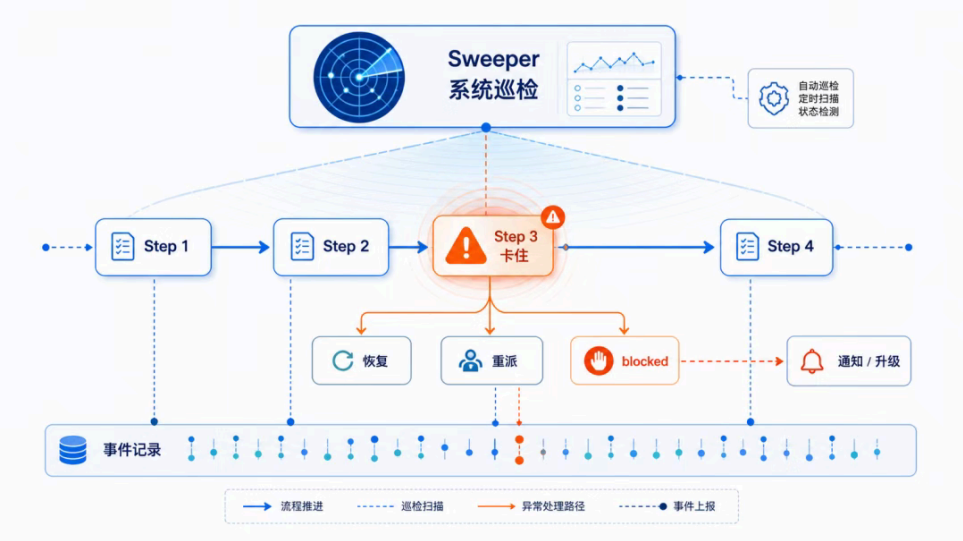
比如 Runtime 掉线、Task 长时间 running、节点没有成功派发、Agent blocked 后没人处理。这些都是真实运行里一定会遇到的问题。

因此，系统里需要巡检和自愈机制。

第一阶段我们补了 blocked、timeout、Sweeper 等能力，主要处理三类静默失败：

场景	处理方式
Runtime 与 Task 异常	掉线标记、长时间 running 重置、孤立任务重分发
调用链不一致	活跃 Step 没有对应 Agent Task 时重新激活
Agent 主动阻塞	暂停流程并通知相关人

这类能力的目标，不是让系统永不失败，而是让失败可见、可恢复、可交给人处理。



4.5 错误信息不足：外部 Agent 失败要能排查

当 Agent 协作链路开始变长，另一个现实问题会变得很明显：失败本身不可怕，可怕的是不知道为什么失败。

比如外部 Agent 执行失败、CLI 调用失败、MCP 工具失败、Runtime 异常、外部回调失败，如果系统只给出一个笼统的 "task failed"，排查成本会非常高。我们无法判断这是 Agent 能力问题、工具调用问题、权限问题、网络问题，还是输入上下文不完整。

所以后续要继续增强错误信息和问题诊断能力。至少要能记录：

- 失败发生在哪个 workflow run step agent task；
- 调用了哪个 Agent 或外部工具；
- 失败类型是什么；
- failure reason 是什么；
- 是否有 stderr / stdout 摘要；
- 是否发生过重试；
- 最终状态是 fail、blocked，还是等待人工处理。

这一类能力的价值是让 Agent 协作系统不仅能执行，也能被排查。否则链路越长，失败越难定位，最终还是会回到"人凭经验猜"。

4.6 做完一批后如何优化：指标、看板和持续改进

如果每个人自己用 Agent，Agent 做不了的时候，人很可能自己补掉。结果是系统看不到：

```
哪个 Agent 经常失败?
哪个节点经常 blocked?
哪类工作适合 Agent?
哪些失败是能力问题?
哪些失败是流程设计问题?
```

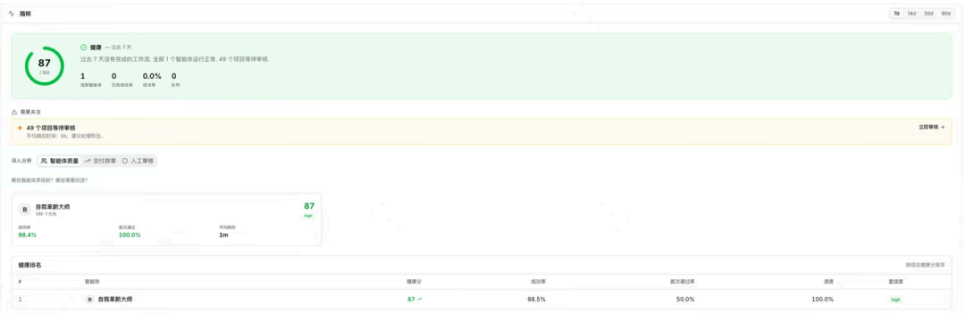
现在工作流运行过程进入系统，指标就可以从事件中自然沉淀出来。

每个 Step、Agent Task、Verdict、Acceptance、Rework 都是事件来源。它们可以形成 Agent 表现、工作流质量、异常分布和场景适配度等指标。

指标层级	关注问题	示例指标
Agent 层	哪些 Agent 稳定	调用次数、成功率、blocked 率、超时率
工作流层	哪条链路有效	完成率、一次通过率、平均耗时、返工率
节点层	哪一步最容易卡	blocked 节点分布、失败原因、验收驳回原因
场景层	哪类工作适合进入	场景通过率、人工介入率、采纳率

指标不是为了做一张好看的 Dashboard，而是为了让 Agent 和工作流的优化不再靠感觉。





阶段小结：第一阶段真正验证了什么

从对象模型、三根骨架，到这六类补出的能力，第一阶段在系统里铺设的东西已经不算少。但比铺了多少能力更重要的，是回头看看这套模式到底验证了什么。

第一阶段的数据还不是长期运营口径下的正式指标，它真正验证的，也不是"做了多少个需求"或者"修了多少个 Bug"。更重要的是，这套模式已经在真实工作里跑起来，并落下来几个关键判断：

- **一段工作可以由系统接手，而不是等人逐步推进。** 外部事件触发后，系统可以创建工作对象并启动流程，工作不再必须等人看到、等人判断、等人手动创建任务才开始往下走。

- **多个 Agent 可以按流程协作。** 评审、分发、执行、通知、验收这些环节，可以由工作流组织 Agent 按节点推进。Agent 不再只是分散的个人工具，而开始成为协作链路中的执行节点。
- **上下文可以在系统里传递。** 通过准出字段和节点模板，上游 Agent 的产物可以被下游节点读取。评审结论、技术方案、实现摘要、自测结果，不再只存在于一次聊天或一次人工转述里。
- **异常可以显式暴露。** blocked、timeout、Sweeper 让流程不会悄悄卡死。Agent 工作流真正进入真实工作，不是看正向链路能不能跑通一次，而是看它卡住时能不能被发现、被记录、被恢复或被交给人处理。
- **执行完成后可以进入验收和返工。** 不是 Agent 回复一句"完成"就结束。系统会通知负责人，进入验收；通过就完成，驳回就回到指定节点重做。

这些判断不是停留在设计层面——它们已经在标准需求、Bug Fix、平台自我迭代、历史问题池等场景里跑出了真实结果，平台也已经集中管理 30+ 个来源各异的 Agent。具体跑了哪些工作，下一章会展开讲。

> **一组 Agent 围绕同一个目标协作完成工作的模式，已经在真实场景里跑起来了。**

而且它跑通的不只是一类工作。

五、除了需求交付，它还能跑哪些工作？

第一阶段最早是从标准需求切入的。

这类工作边界清楚、结果可验收，也已经有对应的 Agent 能力，适合用来验证一条完整链路：从接入、评审、分发、执行，到通知、验收、返工。

但跑起来之后，我们发现，它不只适合需求交付。

只要一件事有明确输入、有处理路径、有结果验收，它就有机会进入同一套 Agent 协作机制。

5.1 标准需求：从进入到验收，一条链路被接住

标准需求是第一批验证场景。

过去，一条需求从进入到完成，通常要经过评审、分发、实现、通知、验收等多个环节。每一步本身未必复杂，但每一步都需要有人看到、有人接手、有人转述上下文、有人推动下一步。

现在，这条链路开始可以被系统接住。

需求进入可执行状态后，平台创建工作对象，启动对应流程。评审 Agent 先看需求是否清晰、合理；分发 Agent 判断模块和执行方向；实现 Agent 根据上下文完成具体改动；最后系统发出完成通知，并进入验收入口。

这不是让某个 Agent 做一个单点任务，而是让一组 Agent 围绕同一条需求接力。

它解决的不是"某一步能不能更快"，而是原来一条需要多个角色持续接力的链路，开始可以在系统里自动进行。

5.2 Bug Fix：测试侧可以把修复闭环跑起来

Bug Fix 的变化更直观。

过去测试发现 Bug 后，通常要提交给研发，等研发看到、理解、修复，再回到测试侧验收。哪怕问题本身不复杂，中间也会有等待、转述和排期。

现在，测试发现问题并提交 Bug 后，可以触发 Agent 修复。Agent 完成处理后，状态回写，测试继续验收。

也就是说，**测试不只是"提出问题并等待修复"，而是可以把一部分 Bug 的修复过程直接闭环起来**：

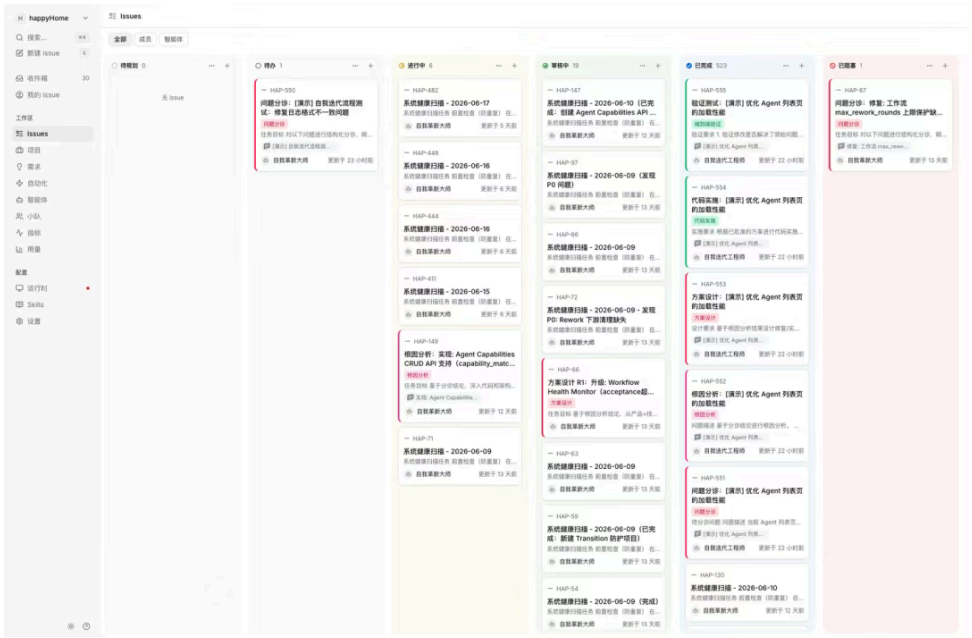
```
测试发现问题
→ 提交 Bug
→ Agent 修复
→ 状态回写
→ 测试验收
```

研发并不是消失，而是从大量逐单处理里释放出来，更多保留在复杂问题、代码边界和兜底判断上。

这个场景很有代表性。因为它让一个过去天然要跨测试和研发两个角色流转的过程，开始在测试侧形成闭环。

5.3 Agent 协作系统：开始反过来建设自己

更有意思的是，这套系统开始"给自己干活"。



它自己的问题、待办和能力缺口，也可以进入同一套机制。Agent 负责分析问题、实施改动、执行验证，最后交给人确认。

过去，平台自身的能力建设也需要人来排期、拆解、开发、验证。现在，一部分平台改进项可以被放进同一套 Agent 协作链路里：

```
发现问题
→ 分析原因
→ 设计方案
→ 实施改动
→ 执行验证
→ 人工确认
```

这不是说系统已经完全自治，而是说明：我们不只是用这套系统处理外部工作，也开始用它建设它自己。

换句话说，平台开始用自己组织起来的 Agent，反过来改进自己的能力。

这件事很重要。因为如果一套 Agent 协作系统只能处理外部任务，它更像一个执行工具；但如果它自己的能力建设也能进入这套机制，它就开始具备持续进化的可能。

5.4 历史问题池：持续消化长期积压项

还有一类工作，过去很容易被忽略：长期存在、有价值，但优先级总是排不上来的问题。

这些问题不一定紧急，但长期存在会影响体验、效率或稳定性。过去它们依赖人工排期，一旦没有明确负责人，就容易沉在问题池里。

现在，这类问题可以被定期拉取，进入评审和处理。适合处理的，由 Agent 执行；完成后，系统通知负责人。

它不一定解决最复杂的问题，但能持续消化那些"重要但不紧急"的长尾优化项。

这类场景的价值不在于单个问题有多大，而在于它改变了过去的处理方式：很多长期挂着的小问题，不再只能等某一次集中排期，而是可以被持续拉起、持续处理、持续清理。

标准需求是一条较完整的协作链路，Bug Fix 是更短的修复闭环，Agent 协作系统自我迭代是用这套机制建设这套机制，历史问题池则是周期性消化长期积压。这四类工作看起来不一样，但底层都有同一个结构：

```
工作进入系统
→ 匹配执行路径
→ Agent 协作处理
→ 结果通知 / 验收 / 返工
→ 状态和指标沉淀
```

这说明我们做的不是某个场景的自动化脚本，而是一种可以迁移的 Agent 协作模式。

它可以接需求，可以接 Bug，可以接平台自己的改进项，也可以接长期积压的问题池。只要一件事能被描述清楚、拆出执行路径、定义完成标准，就有机会进入这套机制。

这也是第一阶段最重要的发现之一：我们不是只跑通了一条需求链路，而是看到了一种更通用的可能——让一组 Agent 围绕同一件事协作，把过去分散在人和人之间的推进过程，放进系统里持续运行。

六、人的位置与协作机制的变化：从处理任务到设计机制

Agent 工作流跑起来以后，人并不会消失，只是位置会上移——从处理单点任务，转向建设让任务能跑起来的机制。

6.1 在单条工作流里，人负责目标、边界和验收

Agent 更适合在明确上下文和约束下完成节点任务。

人更适合处理那些无法完全写进流程里的判断：这件事是否值得做，需求目标是否合理，风险边界在哪里，验收标准是什么，哪些异常必须升级，最终结果能不能接受。

在一条具体工作流里，人更像控制面，Agent 更像执行面，工作流是中间的协调层。

6.2 在能力体系里，人负责建设 Agent、Skill 和工具

组织级 Agent 协作不会自动变强。

它依赖人持续把经验沉淀成能力资产。

比如，一个人反复处理某类配置问题，本质上是在积累一套隐性操作经验。过去这套经验留在个人脑子里；现在它可以被沉淀成专项 Agent、工具调用、规则知识库、工作流模板、准出字段和自动化验证方式。

真正高价值的工作，不是让 Agent 帮自己完成一次任务，而是把某类任务背后的经验变成组织可复用的能力。

6.3 在平台层面，人负责建设可复用运行机制

第一阶段补的很多能力，其实不是为某一个需求服务的。

例如，Fan-out 是复杂任务拆分与收敛的通用能力；Acceptance 是"Agent 完成不等于业务完成"的通用机制；Rework 是"哪里有问题打回哪里"的返工语义；Sweeper 是防止流程静默卡死的自愈机制；Hook / Callback 是外部系统和 Agent 工作流之间的交接协议；运行指标则是后续优化 Agent 和流程的反馈面。

这些能力的价值不在单次执行，而在复用。

人要做的高价值建设，正是把一次次遇到的问题抽象成这样的通用机制。

6.4 在组织层面，人负责选择哪些工作值得进入 Agent 协作模式

并不是所有工作都应该进入 Agent 工作流。

人还需要判断哪些工作高频、低风险、可验证，适合优先进入；哪些工作目标模糊、风险过高，应该继续保留人工主导；哪些 Agent 值得投入长期建设；哪些工具和知识库应该优先沉淀；哪些工作流已经稳定，可以推广到更多团队；哪些工作流效果不好，应该停止或重构。

所以，人不是从 AI 协作系统里退出，而是从处理任务，转向建设任务系统。

6.5 从 Prompt 到 Loop：不是写好一句指令，而是设计一套会自己推进的工作机制

人的位置变了，背后的协作机制也跟着变了。

回头看这次实践，我们做的并不是一个更复杂的 Prompt 系统。

Prompt 代表的是一种典型的个人工作方式：人提出问题，模型给出结果。这个过程很高效，但启动、上下文组织、结果判断和下一步动作，都仍然在人手里。

所以 Prompt 很适合个人单点提效。

但一项组织工作要完成，靠的不是一次输入和一次输出。它需要触发、上下文、执行、反馈、判断、状态更新、下一轮动作和停止条件。

如果说 Prompt Engineering 关注的是：

```
怎么写好一句指令？
```

那么 Loop Engineering 关注的是：

```
怎么设计一套让 Agent 持续推进、验证、纠偏和停止的系统？
```

这两者关注的不是一个层级的问题。普通 coding agent 的 loop，可能是：

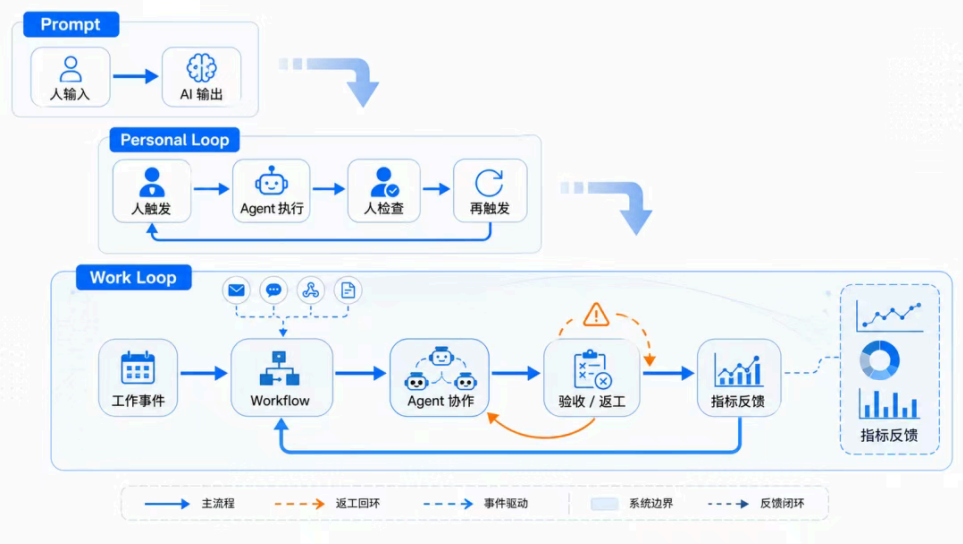
```
修改代码 → 跑测试 → 看错误 → 再修复
```

而我们这次探索的是组织工作里的 loop：

```
工作事件进入
→ 工作流接手
→ Agent 协作执行
→ 状态判断
→ 验收或返工
→ 指标沉淀
→ 下一轮优化
```

所以，这次实践不是把 Prompt 写复杂，而是让一类工作进入一个可触发、可执行、可验收、可返工、可优化的工作循环。

凌晨三点还在交付的，不是某个 Agent，而是一套围绕目标持续运行的 AI 协作系统。



七、下一阶段：把 Agent 协作系统做扎实，再走向 AI 原生工作流

第一阶段是一次必要的过渡：把过去由人类角色接力完成的链路，映射成一条 Agent 工作流。这让落地和验证都更容易，但它还不是终点。

下一阶段的重点不在于继续接更多场景，而是把这套底座做得更稳、更标准、更可观测，再以此为前提，去探索一件更长期的事——如果一项工作从一开始就由 AI 协作完成，它该如何被重新组织。

整体沿三个方向推进。

7.1 把底座做得更稳、更标准

第一阶段证明了"链路能跑起来"，下一阶段要让它"能托付"，也要让它能承载更复杂的 Agent 协作关系。这块我们已经在落地具体方案：

- **工作流引擎升级。** 已经引入 Temporal 并进入测试验证阶段，用它增强长任务执行、状态恢复、失败重试和流程追踪等运行时能力。
- **工作流配置升级。** 结合 Dynamic Config 把当前的表单式配置升级为节点画布，让分支、并行、收敛、返工这类结构能被直观表达和复用。
- **Agent 与平台的标准协议。** 在已有 CLI / MCP 能力基础上继续建设 Multica CLI MCP 和 Agent 标准协议，让状态推进、结果提交、上下文查询等动作不再依赖自然语言"对暗号"，而是有一套统一的接入方式。
- **事件接入与状态同步标准化。** 让需求、Bug、历史问题、指标告警、定时任务等不同来源的工作都能以统一方式进入系统，平台内部的状态也能标准化同步出去。

一句话：让更多事情能进来、更多 Agent 能稳定接进来、更多状态能出去。

7.2 让系统能被观察、能被诊断、能被持续优化

链路变长之后，光看"任务有没有跑完"已经不够。我们需要能回答：哪里跑得好、哪里容易失败、哪些场景适合扩大、哪些节点可能只是过去人类流程的遗留。

这要求两件事：一是把错误信息、调用链路、失败原因、重试记录补完整，让问题能被定位而不是靠人猜；二是把第一阶段沉淀下的运行数据，从"临时拉取"组织成长期可追踪的观测面。

更关键的是让数据反过来指导建设。我们计划建设一个指标分析 Agent，定期回答"哪些 Agent 值得继续投入、哪些场景适合扩大、哪些节点可以合并或前置"这类问题。Agent 协作系统不应该只是"能执行"，它要能被观察、能被诊断、能被持续优化。

7.3 从"把人类流程 Agent 化"走向 AI 原生工作流

这其实就回到了第二章里我们暂时搁置的路径 B——不参考人类流程，从 AI 协作的特性出发重新组织工作。当时不走这条路，是因为没有真实运行数据；而第一阶段攒下的这些数据，正是它的起点。

第一阶段我们做的更多是"把人类流程 Agent 化"：评审、分发、执行、验收，每个节点背后的执行者从人换成了 Agent。这一步很必要，它能快速落地，也保留了原有的责任边界。但它的局限也很清楚——整体仍然是过去的人类组织方式。

真正值得探索的是：如果一项工作从一开始就由 AI 协作完成，它还需要是

```
需求评审 → 任务分发 → 具体实现 → 测试 → 验收
```

还是应该变成更适合 AI 的结构：

```
目标理解 → 约束抽取 → 计划生成 → 工具执行 → 自动验证 → 风险判断 → 人类验收
```

这个问题不能靠拍脑袋回答，要靠真实数据来判断。当我们看清哪些节点最常 blocked、哪些节点经常返工、哪些场景一次通过率最高，就可以反过来回答：哪些节点只是角色分工的遗留？哪些只是为了人类交接而存在？哪些可以由系统直接传递、由规则自动完成？哪些验收必须保留给人？

这就是下一阶段更长期的方向：不是让 Agent 模仿人类角色，而是找到更适合 AI 协作的工作组织方式。



底座的稳定与标准化，正是为了给这一步留出空间。

结语：从超级个体到超级组织

第一阶段还只是开始。

但我们已经看到一个重要变化：AI 不再只是每个角色手里的工具，它开始可以围绕一件事被组织起来，协作推进一段真实工作链路。

过去，我们更多是在打造超级个体：让每个角色借助 AI 变得更强。

现在，我们开始探索超级组织：让一组 Agent 围绕同一个目标协作，把一段原本需要多人接力完成的工作跑下来。

这就是这次第一阶段实践真正验证的东西。

> **不是某个 Agent 变得更强，而是 Agent 开始被组织起来。不是某个人效率更高，而是一段工作链路开始被 AI 协作推进。**

这支凌晨三点还在交付的 AI 军团，离成熟还很远。但它至少让我们看到一种新的可能：那些单点能力已经不弱的"实习生"，正在被组织起来——一起在同一件事上接力、交接、补位。

> AI 进入岗位之后，下一步应该进入协作链路。

等这一步真正走稳了，超级个体手上的那群 Agent，才会被组织成超级组织。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.6万

留言

写留言

还没有留言，来写首评